

Basic Type Systems

JAHEZIA

**College of Science
Northern Border University**



Introduction to Basic type Systems

A type system is a **structured set of rules in a programming language** that assigns specific types to program constructs, such as variables and functions, to ensure safe execution and prevent runtime errors. It defines the permissible values for each type and enforces that operations are performed only on compatible types. By detecting type mismatches, or “type errors,” it helps prevent invalid operations that could lead to program failures.

Type systems are essential in programming languages because they:

- **Prevent errors** by ensuring operations are applied to compatible types.
- **Enable abstraction** by defining data structures and behaviors.
- **Improve program reliability and maintainability.**

Classification of Types

Programming languages classify types into two main categories:

1. Primitive (Basic) Types

- Definition: Built-in, simple types provided by the language.
- Purpose: Represent single, indivisible values.
- Examples:
 1. **Integer (int)** – whole numbers
 2. **Floating-point (float/double)** – decimal numbers
 3. **Boolean (bool)** – true/false values
 4. **Character (char)** – single characters

2. Compound (Composite) Types

- Definition: Types constructed from one or more primitive or other compound types.
- Purpose: Represent structured or complex data.
- Examples:
 1. **Arrays** – collections of elements of the same type
 2. **Tuples** – ordered sets of elements of potentially different types
 3. **Records/Structs** – named fields with different types
 4. **Classes/Objects** – encapsulate data and behavior

Primitive Types

Definition: Basic, built-in data types in a programming language—like integers, floating-point numbers, characters, and booleans—serve as the fundamental building blocks of data. They represent simple, individual values in most programming languages

Key points:

- They store a single value.
- They are usually immutable in most languages.
- Operations on them are simple (like arithmetic, comparisons).

Examples of Primitive Types

TYPE	DESCRIPTION	EXAMPLE
INTEGER (INT)	WHOLE NUMBERS	5, -10, 0
FLOAT (FLOAT)	DECIMAL NUMBERS	3.14, -0.001, 2.0
BOOLEAN (BOOL)	TRUE/FALSE VALUES	TRUE, FALSE
CHARACTER (CHAR)	SINGLE CHARACTERS	'A', 'Z', '#'
STRING (STR)	SEQUENCE OF CHARACTERS (SOMETIMES CONSIDERED PRIMITIVE)	"HELLO", "123"

Compound Types

Definition: Types built from other types—such as arrays, structs (or records), classes, and unions—are used to group multiple values together, which may be of the same or different types

Key points:

- Can hold multiple values.
- Can combine different types of data.
- Often used to model real-world entities or more complex data structures.

Examples of Compound Types

TYPE	DESCRIPTION	EXAMPLE
ARRAY / LIST	ORDERED COLLECTION OF ELEMENTS	[1, 2, 3], ["A", "B"]
TUPLE	FIXED-SIZE, ORDERED COLLECTION	(1, "A", TRUE)
DICTIONARY / MAP	KEY-VALUE PAIRS	{"NAME": "ALICE", "AGE": 25}
SET	UNORDERED COLLECTION OF UNIQUE ELEMENTS	{1, 2, 3}
OBJECT / STRUCT	COMBINATION OF PROPERTIES AND METHODS	CLASS CAR: ...

Static Type System and Dynamic Type systems

1. Static Type system

In a static type system, a variable’s type is determined and verified during compilation, before the program runs. Type checking happens at compile-time. Examples of languages with static typing include Java, C++, and C#.

Key Points:

- Errors are caught before the program runs.
- Variables and expressions must have declared types.
- Safer and optimized code.
- Languages: C, C++, Java, Rust

2. Dynamic Type system

A dynamic type system is one in which type checking is carried out at runtime rather than at compile-time. In such systems, variables do not need explicit type declarations, and the type is associated with the value stored in the variable rather than the variable itself. This means that type verification occurs during program execution, allowing variables to hold values of different types at different times.

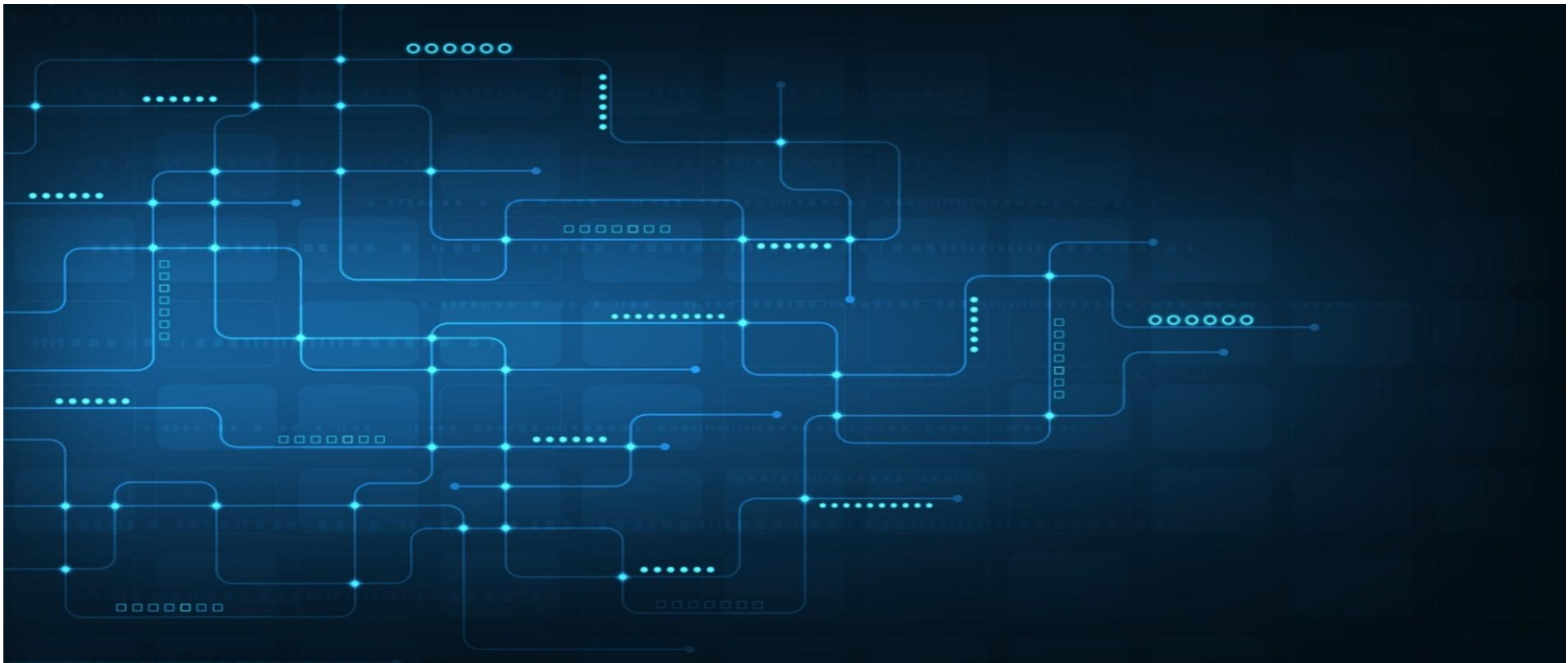
Examples of dynamically typed languages include Python, JavaScript, and Ruby.

Key Points:

- Variables can hold values of different types during execution.
- Errors are caught only when the code is executed.
- More flexible but potentially less safe.
- Languages: Python, JavaScript, Ruby

Comparison table for Static and Dynamic Type Systems

FEATURE	STATIC TYPE SYSTEM	DYNAMIC TYPE SYSTEM
TYPE CHECKING TIME	COMPILE-TIME	RUNTIME
ERROR DETECTION	EARLY, BEFORE EXECUTION	ONLY WHEN EXECUTED
FLEXIBILITY	LESS FLEXIBLE	MORE FLEXIBLE
PERFORMANCE	USUALLY FASTER	SLIGHTLY SLOWER
EXAMPLES	C, C++, JAVA	PYTHON, JAVASCRIPT, RUBY



Program Errors Detected by Type Systems

A type system helps identify errors in a program by making sure operations are performed on compatible types. These errors can be detected either at compile-time, in statically typed languages, or at runtime, in dynamically typed languages. Type systems are designed to catch and prevent specific kinds of errors, mainly those related to incorrect use of data types.

Main types of errors detected by type systems:

- 1. Type Mismatch Errors:** This common error arises when an operation or function expects a value of a certain type but is given a value of an incompatible type. It happens whenever operations are attempted on values with incompatible types.
- 2. Invalid Operations on Data Structures:** Type systems help ensure that operations performed on data structures are valid for their defined types.

3. Undeclared Variable Usage

Using a variable before declaring its type.

4. Invalid Function Arguments

Passing the wrong type of argument to a function.

5. Undefined Variable or Function Usage:

While often caught by the compiler or interpreter as a separate error, type systems can also contribute to detecting the use of undeclared variables or functions, as they lack a defined type.

6. Violation of Data Structure Invariants:

In more advanced type systems (e.g., those with dependent types or advanced static analysis), it's possible to enforce more complex invariants related to data structures, preventing errors that violate these rules.

7. Return Type Mismatch

Returning a value of the wrong type from a function.

8. Null Reference or Dereferencing Errors

Attempting to use a null or uninitialized object.

1.Type Mismatch Errors:

These occur when a value of one type is assigned to a variable of an incompatible type.

```
public class TypeMismatchExample {
    public static void main(String[] args) {
        int num = "Hello"; // ❌ Error: incompatible types: String cannot be
        converted to int
    }
}
```

Explanation:

The type system prevents assigning a String to an int variable.

2. Invalid Operations and Data Structures:

Occurs when you perform an **operation on a data structure** (like arrays, lists, stacks, maps, etc.) that violates type safety or structural rules — not just casting.

```
import java.util.*;

public class Example {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>();
        numbers.add("Hello"); // ❌ Invalid operation: String not allowed in
        List<Integer>
    }
}
```

Explanation:

This is a *type system* error involving data structures — not casting, but misusing a collection's declared type.

3. Undeclared Variable Usage:

Undeclared variable usage means trying to use a variable before it has been declared (i.e., before it's defined with a type and name).

Example :

```
public class UndeclaredVariableExample {
    public static void main(String[] args) {
        x = 10; // ❌ Error: cannot find symbol
        int x = 5;
        System.out.println(x);
    }
}
```

Compiler Error :

error: cannot find symbol

```
x = 10;
^
```

symbol: variable x

Explanation:

The variable x is used before it's declared. Java doesn't allow forward references like this.

4.Invalid Function Arguments:

In Java, **invalid function arguments** occur when the values passed to a method (function) **don't match the expected parameter types, count, or constraints** defined in the method's signature.

Java detects most of these **at compile time** through its **type system**.

```
public class InvalidFunctionArgumentExample {  
    // Method expects an integer parameter  
    public static void printSquare(int number) {  
        System.out.println(number * number);  
    }  
  
    public static void main(String[] args) {  
        printSquare("10"); // ❌ Invalid argument: String cannot be  
converted to int  
    }  
}
```

Explanation:

The method `printSquare(int number)` expects an integer.
The call `printSquare("10")` passes a String, not an int.
The compiler detects this mismatch and gives the error:

5. Undefined Function Usage:

Occurs when a variable or method is **used without being declared or defined**.
Detected **at compile time**.

Example :

```
public class Example {  
    public static void main(String[] args) {  
        int result = add(5, 3); // ❌ add() not defined  
        System.out.println(total); // ❌ total not declared  
    }  
}
```

Compiler Error :

error: cannot find symbol

Explanation:

The compiler doesn't recognize `add` or `total` — they must be declared or defined before use.

6. Violation of Data Structure Invariants :

A **data structure invariant** is a condition that must always hold true for the structure to remain valid (e.g., order in a sorted list, balance in a tree, valid indices in an array).

A **violation** happens when program operations break these rules — often due to logic or type errors.

```
import java.util.PriorityQueue;
```

```
public class InvariantViolationExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(10);
        pq.add(null); // ❌ Violates invariant: null elements not allowed
    }
}
```

Error at runtime :

Exception in thread "**main**" java.lang.NullPointerException

Explanation:

Violation of data structure invariants means performing an operation that breaks the internal rules or consistency of a data structure (e.g., adding null to a PriorityQueue, removing from an empty stack, or corrupting order in a sorted list).

7. Return Type Mismatch:

Occurs when a method's **declared return type** and the **actual returned value's type** don't match. Detected **at compile time** by Java's type system.

Example :

```
public class ReturnTypeMismatchExample {
    public static int getValue() {
        return "Hello"; // ❌ Error: incompatible types: String cannot be
        converted to int
    }

    public static void main(String[] args) {
        System.out.println(getValue());
    }
}
```

Explanation:

Return type mismatch means the method returns a value whose type doesn't match its declared return type — Java catches this at compile time.

8. Null Reference or Dereferencing Errors :

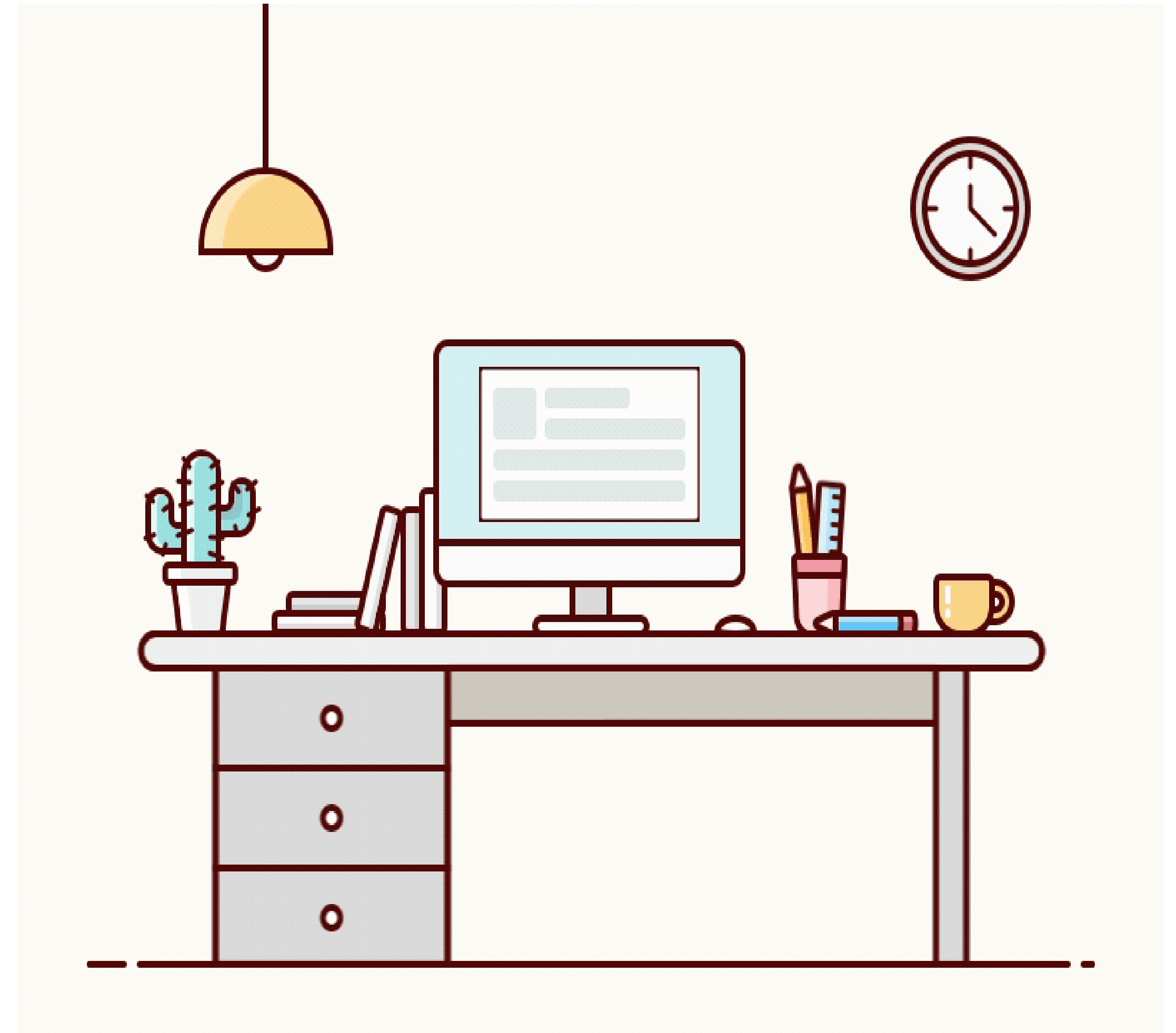
These occur when a program tries to access or call a method/field on a variable that currently refers to null — meaning it doesn't point to any object in memory.

This leads to a **NullPointerException** at runtime.

```
public class NullReferenceExample {  
    public static void main(String[] args) {  
        String name = null;  
        System.out.println(name.length()); // ❌ NullPointerException  
    }  
}
```

Explanation:

A null reference or dereferencing error happens when you try to use a null object reference — Java throws a **NullPointerException** at runtime.



4. Program properties checked statically, and program properties checked dynamically in several languages.

Statically Checked Properties:

Definition: Properties of a program that are verified at compile time without executing the program. Static checking is usually done by the type system or compiler.

Examples of properties checked statically:

PROPERTY	EXAMPLE	LANGUAGE
TYPE CORRECTNESS	ASSIGNING A STRING TO AN INT	JAVA, C#, C++
UNDECLARED VARIABLES	USING X BEFORE DECLARING IT	JAVA, C, C++
METHOD SIGNATURE MISMATCH	CALLING ADD(INT, INT) WITH WRONG TYPE	JAVA, C#, SCALA
ARRAY BOUNDS (SOMETIMES)	FIXED-SIZE ARRAY INDEX IN C (PARTIAL)	C, C++
CONST CORRECTNESS	MODIFYING A CONST VARIABLE	C++

Dynamically Checked Properties:

Definition: Properties that are verified at runtime during program execution. Dynamic checking catches errors that cannot be determined at compile time.

Examples of properties checked Dynamically:

PROPERTY	EXAMPLE	LANGUAGE
NULL REFERENCES	ACCESSING NULL.LENGTH()	JAVA, C#, PYTHON
ARRAY BOUNDS	ACCESSING ARR[10] IF ARRAY HAS 5 ELEMENTS	JAVA, PYTHON
DIVISION BY ZERO	INT A = 10 / 0	JAVA, PYTHON, C#
TYPE CASTING AT RUNTIME	(INTEGER) OBJ WHERE OBJ IS A STRING	JAVA
MEMORY SAFETY CHECKS	DANGLING POINTERS	PYTHON, JAVA (MANAGED MEMORY)

Comparison Table: Static vs Dynamic Checking



ASPECT	STATIC CHECKING	DYNAMIC CHECKING
WHEN	COMPILE-TIME	RUNTIME
WHAT IS CHECKED	TYPE CORRECTNESS, DECLARATION, METHOD SIGNATURES, VISIBILITY, INHERITANCE	NULL REFERENCES, ARRAY BOUNDS, TYPE CASTING, DIVISION BY ZERO, MEMORY SAFETY
DETECTED BY	COMPILER / TYPE SYSTEM	RUNTIME SYSTEM / INTERPRETER
LANGUAGES	JAVA, C#, C++, SCALA	JAVA, PYTHON, C#, JAVASCRIPT
ADVANTAGES	EARLY ERROR DETECTION, SAFER, COMPILER OPTIMIZATIONS	DETECTS RUNTIME/DATA-DEPENDENT ERRORS, DYNAMIC SAFETY
LIMITATIONS	CANNOT DETECT DATA-DEPENDENT ERRORS	SLOWER, ERRORS FOUND LATE

Cross-Language Examples

LANGUAGE	STATIC CHECKING EXAMPLES	DYNAMIC CHECKING EXAMPLES
JAVA	TYPE MISMATCH, UNDECLARED VARIABLES, WRONG METHOD ARGS	NULLPOINTEREXCEPTION, ARRAYINDEXOUTOFBOUNDS, CLASSCASTEXCEPTION
C#	TYPE MISMATCH, WRONG METHOD SIGNATURE, PRIVATE ACCESS	NULLREFERENCEEXCEPTION, RUNTIME CASTING ERRORS
C++	TYPE MISMATCH, CONST CORRECTNESS, UNDECLARED VARIABLES	UNDEFINED BEHAVIOR, DANGLING POINTERS (NO BUILT-IN RUNTIME CHECKS)
PYTHON	MINIMAL (ONLY SYNTAX)	TYPE ERRORS AT RUNTIME, INDEXERROR, ATTRIBUTEERROR, DIVISION BY ZERO
JAVASCRIPT	MINIMAL (SYNTAX)	TYPE COERCION ERRORS, RUNTIME REFERENCE ERRORS

Generics, subtyping and overloading

1. **Generics** : Generics allow you to write classes, interfaces, and methods with type parameters, so they can work with any type while still being type-safe. They were introduced in Java 5 to reduce the need for casting and runtime type errors.

// Generic class

```
class Box<T> {  
    private T content;  
  
    public void set(T content) { this.content = content; }  
    public T get() { return content; }  
}
```

```
public class GenericsExample {  
    public static void main(String[] args) {  
        Box<Integer> intBox = new Box<>();  
        intBox.set(10);  
        System.out.println(intBox.get()); // 10  
  
        Box<String> strBox = new Box<>();  
        strBox.set("Hello");  
        System.out.println(strBox.get()); // Hello  
    }  
}
```

Key Points

- <T> is a type parameter.
- Prevents ClassCastException at runtime.
- Works with subtyping carefully, since Box<Cat> is not a subtype of Box<Animal> (invariant).

2. Subtyping

Subtyping is the relationship where one type is considered a subtype of another.

- In Java, class Cat extends Animal → Cat is a subtype of Animal.
- Allows polymorphism.

```
class Animal { void speak() { System.out.println("Animal sound"); } }  
class Dog extends Animal { void speak() { System.out.println("Woof!"); } }
```

```
public class SubtypingExample {  
    public static void main(String[] args) {  
        Animal a = new Dog(); // ✅ Dog is a subtype of Animal  
        a.speak(); // Woof!  
    }  
}
```

Important Notes

- Subtyping works naturally for classes and interfaces.
- But generics are invariant by default: List<Dog> is NOT a subtype of List<Animal>.
- You can use wildcards to allow covariance or contravariance:



3. Method Overloading

Overloading occurs when multiple methods have the same name but different parameter types or counts in the same class.

```
class MathUtil {  
    public int add(int a, int b) { return a + b; }  
    public double add(double a, double b) { return a + b; }  
}  
  
public class OverloadingExample {  
    public static void main(String[] args) {  
        MathUtil m = new MathUtil();  
        System.out.println(m.add(5, 10)); // calls add(int,int)  
        System.out.println(m.add(5.0, 10.0)); // calls add(double,double)  
    }  
}
```

Important Notes:

- Overloading is resolved at compile-time based on method signature.
- Return type alone cannot distinguish overloaded methods.
- Generics + overloading can lead to compiler ambiguity, so be careful.

